

Cellule de code

```
print("-"*70)
print("Exercice 01- Prise en main et vérification de l'environnement")
print("-"*70)
print("#Version de TensorFlow")
print(f"Version de TensorFlow : {tf.version}")
print("#Version de Keras")
print(f"Version de Keras : {keras.version}")
print("#Vérification du GPU")
gpu_available = tf.config.list_physical_devices('GPU')
if gpu_available:
    print("Le GPU est disponible.")
else:
    print("Le GPU n'est pas disponible.")
print("#Impact de l'absence de GPU sur l'entraînement d'un modèle")
print("""Réponse:
    L'absence de GPU signifie que l'environnement,
    Google Colab utilisera exclusivement le CPU
    (processeur central) pour effectuer les calculs.
    Contrairement au CPU, qui est conçu pour gérer des tâches
    séquentielles complexes, le GPU est optimisé pour le calcul
    parallèle.
    Comme l'entraînement des modèles de Deep Learning repose massivement
    sur
    des opérations matricielles simultanées,
    le GPU est l'outil idéal pour ces tâches.""")
print("-"*70)
print("Exercice 2- Analyse critique du résultat d'entraînement ")
print("-"*70)
print("1. Qu'est-ce que la MSE (Mean Squared Error) ?")
print("""Réponse:
    La MSE (Erreur Quadratique Moyenne) est une métrique
    utilisée pour mesurer la performance d'un modèle de régression.
    Elle calcule la moyenne des carrés des écarts entre les valeurs
    réelles et les prédictions du modèle.""")
print("""sa formule mathématique:
    MSE = (1/n) * Σ(y_i - ŷ_i)²""")
print("""    n : nombre d'observation
    y_i : valeur réelle
    hat{y}_i : valeur prédite par le modèle""")
print("""2. Pourquoi une valeur de MSE égale à 2.317 est-elle
    considérée comme anormalement élevée pour des données normalisées
    entre 0 et 1 ?""")
print("""Réponse: """)
print("""Dans le contexte de données normalisées entre 0 et 1,
    l'erreur maximale possible pour un seul échantillon est de 1
    (soit |1 - 0| = 1), et par conséquent, le carré de cette erreur est
    également de 1.
    Puisque la MSE est la "moyenne" de ces carrés, elle devrait
    logiquement être
    inférieure à 1. Une valeur de 2.317 indique que l'erreur dépasse la
    plage totale des données,
    ce qui signale un dysfonctionnement majeur dans le modèle ou dans le
    processus d'entraînement.""")
print("""3. Quelle valeur de MSE serait considérée comme acceptable
    pour un modèle bien entraîné sur ces données ?""")
print("""Pour des données comprises entre 0 et 1, une valeur acceptable
    doit être très proche de zéro.
```

Cellule de code

On recherche généralement des valeurs telles que 0.01, 0.001 ou moins, selon la précision requise.

Plus la valeur tend vers zéro, plus les prédictions sont fiables."""

```
print("""4. Hypothèse sur les causes probables de cette valeur élevée avant toute modification du programme :""")
```

```
print("""Plusieurs raisons peuvent expliquer ce résultat :""")
```

```
print("""Erreur de normalisation :""")
```

```
print("""Les étiquettes (Labels) cibles n'ont peut-être pas été normalisées
```

```
et sont restées dans leur plage d'origine, alors que les sorties du modèle
```

```
sont limitées entre 0 et 1.""")
```

```
print("""Fonction d'activation inappropriée :""")
```

```
print("""L'utilisation d'une fonction dans la couche de sortie qui ne correspond pas à la plage des données (par exemple, une fonction produisant des valeurs négatives ou supérieures à 1""")
```

```
print("""Taux d'apprentissage (Learning Rate) trop élevé :""")
```

```
print("""Cela peut provoquer une divergence du modèle au lieu d'une convergence,
```

```
entraînant une "explosion" de la perte (loss)""")
```

```
print("""Erreur dans le code de la fonction de perte: """)
```

```
print("""Un bug algorithmique dans le calcul du coût qui multiplierait anormalement les résultats.""")
```

```
print("-"*70)
```

```
print("Exercice 03- Extraction de l'équation du modèle")
```

```
print("-"*70)
```

```
print("Préparation des données (Normalisation)")
```

```
import numpy as np
```

```
print("Données d'entraînement (Surface et Prix)")
```

```
X_train = np.array([[50], [80], [120], [150]], dtype=float)
```

```
y_train = np.array([[100], [160], [240], [300]], dtype=float)
```

```
print("Calcul des paramètres de normalisation (Moyenne et Écart-type)")
```

```
X_train_mean = np.mean(X_train)
```

```
X_train_std = np.std(X_train)
```

```
y_train_mean = np.mean(y_train)
```

```
y_train_std = np.std(y_train)
```

```
print("Application de la normalisation (Standardisation)")
```

```
X_train_norm = (X_train - X_train_mean) / X_train_std
```

```
y_train_norm = (y_train - y_train_mean) / y_train_std
```

```
print("Données préparées avec succès !")
```

```
print("Construction et entraînement du modèle")
```

```
from tensorflow.keras.models import Sequential
```

```
from tensorflow.keras.layers import Dense
```

```
print("Construction du modèle")
```

```
model = Sequential([
```

```
    Dense(1, input_shape=[1])
```

```
])
```

```
print("Compilation du modèle")
```

```
model.compile(optimizer='adam', loss='mean_squared_error')
```

```
print("Entraînement du modèle")
```

```
model.fit(X_train_norm, y_train_norm, epochs=100)
```

```
print("Extraction des poids (w) et du biais (b)")
```

```
weights = model.get_weights()
```

```
w_norm = weights[0][0][0]
```

```
b_norm = weights[1][0]
```

```
print(f"L'équation normalisée: prix_norm = {w_norm:.4f} * surface_norm + {b_norm:.4f}")
```

```
print("Dénormalisation (Retour à l'échelle réelle)")
```

Cellule de code

```
print("Calcul des coefficients réels")
w_real = w_norm * (y_train_std / X_train_std)
b_real = (b_norm * y_train_std) + y_train_mean - (w_real *
X_train_mean)
print(f"L'équation réelle: prix = {w_real:.4f} * surface +
{b_real:.4f}")
print("Vérification manuelle pour une surface de 100 m²")
surface_test = 100
prix_predit = (w_real * surface_test) + b_real
print(f"Pour une surface de {surface_test} m²:")
print(f"Le prix prédit est : {prix_predit:.2f} k€")
print("-"*70)
print("Exercice 04- Diagnostic d'un probleme de convergence")
print("1. Analyse des résultats et interprétation")
print("""
En comparant les Prédictions et les Vérités terrain, on observe les
faits suivants :
Écart de signe : Les valeurs réelles sont comprises entre 0 et 1
(positives),
    tandis que les prédictions sont toutes négatives.
Divergence : Au lieu de se rapprocher des valeurs cibles, les
prédictions s'en
éloignent de plus en plus (elles passent de -0.1458 à -0.6326).
Conclusion : Le modèle ne converge pas. Il est en état de divergence.
L'erreur
    (loss) augmente à chaque étape au lieu de diminuer.
""")
print("2. Identification des deux paramètres responsables")
print("""Les deux paramètres du programme initial
    qui causent généralement ce dysfonctionnement sont :
Le taux d'apprentissage (Learning Rate) :
Il est très probablement trop élevé. Un pas trop grand empêche
l'algorithme de
    trouver le minimum de la fonction de perte et le fait "rebondir" de
manière
    instable vers l'infini (ou des valeurs aberrantes).
L'initialisation des poids (Weights Initialization) : Si les poids de
départ
    sont mal calibrés ou trop grands, combinés à un mauvais learning rate,
cela précipite la divergence du modèle dès les premières itérations.
""")
print("3. Relation entre le taux d'apprentissage et la vitesse de
convergence")
print("""Le taux d'apprentissage (learning rate) détermine la taille
des pas effectués pour atteindre le minimum de la fonction de perte :
Trop faible : La convergence est très lente. Le modèle mettra
énormément de temps (trop d'époques) pour apprendre.
Optimal : Le modèle atteint le minimum de manière rapide et stable.
Trop élevé : Le modèle dépasse le minimum, oscille, ou diverge
complètement (comme illustré dans vos résultats)
""")
print("4. Définition de la convergence et détection par la 'Loss' ")
print("""Qu'est-ce que la convergence ?
C'est l'état où le modèle a fini d'apprendre. Cela signifie que les
poids ont été ajustés de façon à minimiser l'erreur, et que les
prédictions sont les plus proches possibles des vérités terrain.
Comment la détecter avec la "Loss" ?
On utilise une courbe de perte (Loss Curve) :
```

Cellule de code

Convergence : La courbe descend progressivement et finit par stagner (atteindre un plateau) près de zéro.
Divergence (votre cas) : La courbe de perte augmente brutalement vers l'infini, indiquant que le modèle s'égare

```
"""
print("-"*70)
print("Exercice 05- Correction et optimisation du programme ")
print("-"*70)
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.optimizers import Adam
print("1.Préparation des données (Exemple illustratif)")
X_train = np.linspace(0, 10, 100).reshape(-1, 1)
y_train = 2 * X_train + 1 + np.random.randn(100, 1)
print("2. Modification et Amélioration du Programme (Exercice 5)")
print("Création du modèle")
model = Sequential([
    Dense(10, activation='relu', input_shape=(1,)),
    Dense(1)
])

print("MODIFICATION : Learning Rate à 0.1")
print("JUSTIFICATION : Un taux de 0.1 accélère la convergence vers le
minimum de la fonction de perte.")
custom_optimizer = Adam(learning_rate=0.1)
model.compile(optimizer=custom_optimizer, loss='mse')
print("MODIFICATION : Augmenter les Epochs à 500")
# Callback pour afficher la perte toutes les 100 époques
class LossDisplayCallback(tf.keras.callbacks.Callback):
    def on_epoch_end(self, epoch, logs=None):
        if (epoch + 1) % 100 == 0:
            print(f"Époque {epoch + 1}: Perte (Loss) =
{logs['loss']:.4f}")

print("Démarrage de l'entraînement...")
history = model.fit(
    X_train,
    y_train,
    epochs=500,
    verbose=0,
    callbacks=[LossDisplayCallback()]
)
print("3. Visualisation et Analyse")
plt.figure(figsize=(10, 5))
plt.plot(history.history['loss'], color='red', linewidth=2)
plt.title('Évolution de la Perte sur 500 Époques')
plt.xlabel('Époques')
plt.ylabel('Perte (Loss)')
plt.grid(True, linestyle='--', alpha=0.6)
plt.show()
print("4. Réponses Théoriques (À copier dans votre compte-rendu)")
print("\n" + "="*60)
print("Réponses aux questions de l'exercice :")
print("1. Justification du Learning Rate (0.1) :")
```

Cellule de code

```
print("    Ce choix permet des mises à jour plus importantes des poids,
ce qui aide")
print("    le modèle à apprendre plus rapidement au début de
l'entraînement.")
print("\n2. Pourquoi 500 époques ? :")
print("    Un nombre plus élevé d'époques permet à l'algorithme
d'affiner les poids")
print("    et d'atteindre une convergence plus stable et précise.")
print("\n3. Commentaire sur l'évolution de la perte (1 à 500) :")
print("    On observe une chute drastique de la perte lors des premières
époques,")
print("    suivie d'une stabilisation (plateau) vers la 500ème époque,
indiquant")
print("    que le modèle a atteint son point d'apprentissage optimal.")
```

```
print("-" * 70)
print("Exercice 06 - Interprétation des résultats finaux")
print("-" * 70)
```

```
import numpy as np
import pandas as pd
from sklearn.linear_model import LinearRegression
import matplotlib.pyplot as plt

# 1. Préparation des données (Structure parfaitement linéaire)
print("1. Préparation des données (Structure parfaitement linéaire)")
print("Équation : Prix = 2.0000 * Surface + 0.00")

np.random.seed(42)
surface = np.linspace(50, 500, 100).reshape(-1, 1) # Surface en m²
prix = 2.0000 * surface + 0.00 # Prix en (k€)

# 2. Construction et exécution du modèle
print("2. Construction et exécution du modèle")
model = LinearRegression()
model.fit(surface, prix)

# CORRECTION : Extraction des valeurs en tant que scalaires (nombres)
coef = model.coef_[0][0] if isinstance(model.coef_[0], np.ndarray) else
model.coef_[0]
intercept = model.intercept_[0] if isinstance(model.intercept_,
np.ndarray) else model.intercept_

# Calcul du MSE
predictions = model.predict(surface)
loss = np.mean((predictions - prix)**2)

print("\nAffichage des Résultats")
print(f"## Résultats du Modèle ##")
# Utilisation de float() pour garantir la compatibilité avec le
formatage .4f
print(f"Coefficient (Prix/m²): {float(coef):.4f} k€")
print(f"Ordonnée à l'origine (Intercept): {float(intercept):.2f}")
print(f"Valeur de la perte finale (Loss): {float(loss):.6f}")

print("\nRéponses aux questions théoriques de l'exercice :")
```

Cellule de code

```
print("""
1. Pourquoi ce résultat est attendu mathématiquement ?
   Le résultat est prévisible car la structure des données est
   "parfaitement linéaire".
   Le modèle retrouve exactement les paramètres originaux car il n'y a
   aucun bruit.

2. Signification de la perte 0.000000 et est-elle souhaitable ?
   Cela signifie un "ajustement parfait". En situation réelle, ce n'est
   pas
   souhaitable car cela indique souvent des données artificielles ou un
   risque de "Surapprentissage" (Overfitting).

3. Limites du modèle face à des données immobilières réelles :
   - Absence d'autres variables (emplacement, état du bâtiment).
   - Les relations réelles ne sont pas toujours strictement linéaires.

4. Propositions d'amélioration :
   A. Utiliser une "Régression Multiple" (ajouter la localisation,
   l'âge, etc.).
   B. Utiliser une "Régression Polynomiale" pour les relations courbes.
""")

# Visualisation graphique
print("Visualisation graphique...")
plt.figure(figsize=(10, 6))
plt.scatter(surface, prix, color='blue', alpha=0.5, label='Données
Réelles (Simulées)')
plt.plot(surface, predictions, color='red', linewidth=2, label='Modèle
Linéaire')
plt.title('Relation entre Surface et Prix (Structure Linéaire
Parfaite)')
plt.xlabel('Surface (m²)')
plt.ylabel('Prix (k€)')
plt.legend()
plt.grid(True, linestyle='--', alpha=0.7)
plt.show()

print("-" * 70)
def generate_final_report():
    print("="*80)
    print("EXERCICE 07 - SYNTHÈSE ET RÉFLEXION CRITIQUE : RAPPORT
D'APPRENTISSAGE")
    print("="*80)

    # 1. Normalisation des données
    print("\n1. Normalisation des données : Pourquoi et Comment ?")
    print("-" * 55)
    print("    * Pourquoi : Pour unifier les échelles des entrées, ce
qui empêche les")
    print("        caractéristiques à grandes valeurs de dominer et
accélère la")
    print("        convergence de l'algorithme de 'Descente de
Gradient'.")
    print("    * Comment : Nous avons utilisé la Standardisation en
soustrayant la")
    print("        moyenne (Mean) et en divisant par l'écart-type (Std).")
```

Cellule de code

```
# 2. Fonction de perte MSE
print("\n2. Fonction de perte MSE : Définition et Interprétation")
print("-" * 55)
print("    * Définition : La MSE (Mean Squared Error) est la moyenne
des carrés des")
print("        écarts entre les valeurs réelles et les prédictions.")
print("    * Interprétation : Elle indique la précision du modèle ;
plus elle est")
print("        proche de zéro, meilleur est le modèle. Une valeur de
2.317 signalait")
print("        un échec puisque l'erreur ne devrait pas dépasser 1
avec des données normalisées.")

# 3. Rôle des Hyperparamètres
print("\n3. Rôle des Hyperparamètres")
print("-" * 55)
print("    * Taux d'apprentissage (Learning Rate) : Définit la
taille du pas lors de")
print("        la mise à jour des poids. Nous avons choisi 0.1 pour
garantir une")
print("        vitesse d'apprentissage sans divergence.")
print("    * Nombre d'époques (Epochs) : Représente le nombre de
passages du modèle")
print("        sur les données. Passer à 500 époques a permis au
modèle de se")
print("        stabiliser et d'atteindre l'erreur minimale.")

# 4. Extraction des paramètres du modèle
print("\n4. Extraction des paramètres du modèle de régression
linéaire")
print("-" * 55)
print("    * Le poids (w) et le biais (b) ont été extraits de la
couche 'Dense'.")
print("    * Nous avons effectué une 'Dénormalisation' pour
convertir ces paramètres")
print("        en valeurs réelles (Surface et Prix) afin d'obtenir une
équation applicable.")

# 5. Interprétation des prédictions et évaluation des performances
print("\n5. Interprétation des prédictions et évaluation des
performances")
print("-" * 55)
print("    * Prédications : Le modèle a réussi à prédire avec
précision (ex: prix")
print("        d'une maison de 100 m²).")
print("    * Évaluation : Via la 'courbe de perte', nous avons
observé une chute")
print("        progressive suivie d'un plateau, confirmant la
convergence du modèle.")

print("-" * 55)
print("FIN DU RAPPORT")
print("-" * 55)

# Exécution de la fonction
generate_final_report()
```

Résultat d'exécution

Exercice 01- Prise en main et vérification de l'environnement

```
#Version de TensorFlow
Version de TensorFlow : <module 'tensorflow._api.v2.version' from
'/usr/local/lib/python3.12/dist-
packages/tensorflow/_api/v2/version/__init__.py'>
#Version de Keras
Version de Keras : <function version at 0x7beafbb20180>
#Vérification du GPU
Le GPU n'est pas disponible.
#Impact de l'absence de GPU sur l'entraînement d'un modèle
Réponse:
L'absence de GPU signifie que l'environnement,
Google Colab utilisera exclusivement le CPU
(processeur central) pour effectuer les calculs.
Contrairement au CPU, qui est conçu pour gérer des tâches
séquentielles complexes, le GPU est optimisé pour le calcul parallèle.
Comme l'entraînement des modèles de Deep Learning repose massivement sur
des opérations matricielles simultanées,
le GPU est l'outil idéal pour ces tâches.
```

Exercice 2- Analyse critique du résultat d'entraînement

1. Qu'est-ce que la MSE (Mean Squared Error) ?

Réponse:
La MSE (Erreur Quadratique Moyenne) est une métrique
utilisée pour mesurer la performance d'un modèle de régression.
Elle calcule la moyenne des carrés des écarts entre les valeurs
réelles et les prédictions du modèle.

sa formule mathématique:

$$MSE = (1/n) * \sum (y_i - \hat{y}_i)^2$$

n : nombre d'observation
y_i : valeur réelle
hat{y}_i : valeur prédite par le modèle)

2. Pourquoi une valeur de MSE égale à 2.317 est-elle
considérée comme anormalement élevée pour des données normalisées
entre 0 et 1 ?

Réponse:
Dans le contexte de données normalisées entre 0 et 1,
l'erreur maximale possible pour un seul échantillon est de 1
(soit $|1 - 0| = 1$), et par conséquent, le carré de cette erreur est
également de 1.
Puisque la MSE est la "moyenne" de ces carrés, elle devrait logiquement
être
inférieure à 1. Une valeur de 2.317 indique que l'erreur dépasse la plage
totale des données,

ce qui signale un dysfonctionnement majeur dans le modèle ou dans le
processus d'entraînement.

3. Quelle valeur de MSE serait considérée comme acceptable pour un modèle
bien entraîné sur ces données ?

Pour des données comprises entre 0 et 1, une valeur acceptable doit être
très proche de zéro.

On recherche généralement des valeurs telles que 0.01, 0.001 ou moins,
selon la précision requise.

Plus la valeur tend vers zéro, plus les prédictions sont fiables.

4. Hypothèse sur les causes probables de cette valeur élevée avant toute
modification du programme :

Plusieurs raisons peuvent expliquer ce résultat :

Erreur de normalisation :

Les étiquettes (Labels) cibles n'ont peut-être pas été normalisées

Résultat d'exécution

et sont restées dans leur plage d'origine, alors que les sorties du modèle sont limitées entre 0 et 1.

Fonction d'activation inappropriée :

L'utilisation d'une fonction dans la couche de sortie qui

ne correspond pas à la plage des données (par exemple, une fonction produisant des valeurs négatives ou supérieures à 1

Taux d'apprentissage (Learning Rate) trop élevé :

Cela peut provoquer une divergence du modèle au lieu d'une convergence, entraînant une "explosion" de la perte (loss)

Erreur dans le code de la fonction de perte:

Un bug algorithmique dans le calcul du coût qui multiplierait anormalement les résultats.

Exercice 03- Extraction de l'équation du modèle

Préparation des données (Normalisation)

Données d'entraînement (Surface et Prix

Calcul des paramètres de normalisation (Moyenne et Écart-type

Application de la normalisation (Standardisation)

Données préparées avec succès !

Construction et entraînement du modèle

Construction du modèle

Compilation du modèle

Entraînement du modèle

/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106:

UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.

super().__init__(activity_regularizer=activity_regularizer, **kwargs)

Epoch 1/100

1/1 ----- 1s 1s/step - loss: 5.2917e-06

Epoch 2/100

1/1 ----- 0s 110ms/step - loss: 1.6928e-06

Epoch 3/100

1/1 ----- 0s 294ms/step - loss: 1.2388e-07

Epoch 4/100

1/1 ----- 0s 145ms/step - loss: 2.1119e-07

Epoch 5/100

1/1 ----- 0s 101ms/step - loss: 1.0321e-06

Epoch 6/100

1/1 ----- 0s 126ms/step - loss: 1.5980e-06

Epoch 7/100

1/1 ----- 0s 74ms/step - loss: 1.5637e-06

Epoch 8/100

1/1 ----- 0s 159ms/step - loss: 1.1079e-06

Epoch 9/100

1/1 ----- 0s 211ms/step - loss: 5.4588e-07

Epoch 10/100

1/1 ----- 0s 145ms/step - loss: 1.3426e-07

Epoch 11/100

1/1 ----- 0s 117ms/step - loss: 1.7083e-10

Epoch 12/100

1/1 ----- 0s 264ms/step - loss: 1.2351e-07

Epoch 13/100

1/1 ----- 0s 228ms/step - loss: 3.6956e-07

Epoch 14/100

1/1 ----- 0s 233ms/step - loss: 5.7261e-07

Epoch 15/100

1/1 ----- 0s 102ms/step - loss: 6.2658e-07

Résultat d'exécution

Epoch 16/100	1/1	0s	122ms/step	-	loss: 5.2200e-07
Epoch 17/100	1/1	0s	100ms/step	-	loss: 3.2590e-07
Epoch 18/100	1/1	0s	46ms/step	-	loss: 1.3227e-07
Epoch 19/100	1/1	0s	48ms/step	-	loss: 1.7116e-08
Epoch 20/100	1/1	0s	46ms/step	-	loss: 9.4366e-09
Epoch 21/100	1/1	0s	48ms/step	-	loss: 8.5304e-08
Epoch 22/100	1/1	0s	46ms/step	-	loss: 1.8613e-07
Epoch 23/100	1/1	0s	49ms/step	-	loss: 2.5177e-07
Epoch 24/100	1/1	0s	52ms/step	-	loss: 2.5024e-07
Epoch 25/100	1/1	0s	48ms/step	-	loss: 1.8800e-07
Epoch 26/100	1/1	0s	60ms/step	-	loss: 1.0010e-07
Epoch 27/100	1/1	0s	51ms/step	-	loss: 2.8241e-08
Epoch 28/100	1/1	0s	51ms/step	-	loss: 1.1065e-10
Epoch 29/100	1/1	0s	51ms/step	-	loss: 1.8079e-08
Epoch 30/100	1/1	0s	49ms/step	-	loss: 6.1514e-08
Epoch 31/100	1/1	0s	48ms/step	-	loss: 1.0045e-07
Epoch 32/100	1/1	0s	49ms/step	-	loss: 1.1235e-07
Epoch 33/100	1/1	0s	62ms/step	-	loss: 9.2718e-08
Epoch 34/100	1/1	0s	48ms/step	-	loss: 5.4396e-08
Epoch 35/100	1/1	0s	47ms/step	-	loss: 1.8201e-08
Epoch 36/100	1/1	0s	78ms/step	-	loss: 6.5980e-10
Epoch 37/100	1/1	0s	81ms/step	-	loss: 5.9986e-09
Epoch 38/100	1/1	0s	83ms/step	-	loss: 2.5561e-08
Epoch 39/100	1/1	0s	82ms/step	-	loss: 4.4403e-08
Epoch 40/100	1/1	0s	135ms/step	-	loss: 5.0609e-08
Epoch 41/100	1/1	0s	71ms/step	-	loss: 4.1357e-08
Epoch 42/100	1/1	0s	73ms/step	-	loss: 2.3125e-08
Epoch 43/100	1/1	0s	136ms/step	-	loss: 6.6635e-09
Epoch 44/100					

Résultat d'exécution

1/1	0s	63ms/step	- loss: 2.1467e-11
Epoch 45/100			
1/1	0s	64ms/step	- loss: 4.3675e-09
Epoch 46/100			
1/1	0s	99ms/step	- loss: 1.4339e-08
Epoch 47/100			
1/1	0s	63ms/step	- loss: 2.2196e-08
Epoch 48/100			
1/1	0s	63ms/step	- loss: 2.2769e-08
Epoch 49/100			
1/1	0s	88ms/step	- loss: 1.6270e-08
Epoch 50/100			
1/1	0s	127ms/step	- loss: 7.2056e-09
Epoch 51/100			
1/1	0s	138ms/step	- loss: 1.0037e-09
Epoch 52/100			
1/1	0s	149ms/step	- loss: 4.8642e-10
Epoch 53/100			
1/1	0s	135ms/step	- loss: 4.5009e-09
Epoch 54/100			
1/1	0s	155ms/step	- loss: 9.2791e-09
Epoch 55/100			
1/1	0s	73ms/step	- loss: 1.1207e-08
Epoch 56/100			
1/1	0s	136ms/step	- loss: 9.1589e-09
Epoch 57/100			
1/1	0s	62ms/step	- loss: 4.8106e-09
Epoch 58/100			
1/1	0s	73ms/step	- loss: 1.0805e-09
Epoch 59/100			
1/1	0s	72ms/step	- loss: 3.8229e-11
Epoch 60/100			
1/1	0s	153ms/step	- loss: 1.6474e-09
Epoch 61/100			
1/1	0s	138ms/step	- loss: 4.1308e-09
Epoch 62/100			
1/1	0s	72ms/step	- loss: 5.4354e-09
Epoch 63/100			
1/1	0s	62ms/step	- loss: 4.6854e-09
Epoch 64/100			
1/1	0s	134ms/step	- loss: 2.5578e-09
Epoch 65/100			
1/1	0s	46ms/step	- loss: 6.1011e-10
Epoch 66/100			
1/1	0s	50ms/step	- loss: 1.1859e-11
Epoch 67/100			
1/1	0s	56ms/step	- loss: 8.0713e-10
Epoch 68/100			
1/1	0s	49ms/step	- loss: 2.0606e-09
Epoch 69/100			
1/1	0s	49ms/step	- loss: 2.6877e-09
Epoch 70/100			
1/1	0s	48ms/step	- loss: 2.2539e-09
Epoch 71/100			
1/1	0s	48ms/step	- loss: 1.1463e-09
Epoch 72/100			
1/1	0s	48ms/step	- loss: 2.1880e-10

Résultat d'exécution

Epoch 73/100	1/1	0s	47ms/step	-	loss: 3.0704e-11
Epoch 74/100	1/1	0s	48ms/step	-	loss: 5.1942e-10
Epoch 75/100	1/1	0s	50ms/step	-	loss: 1.1410e-09
Epoch 76/100	1/1	0s	51ms/step	-	loss: 1.3462e-09
Epoch 77/100	1/1	0s	50ms/step	-	loss: 1.0037e-09
Epoch 78/100	1/1	0s	51ms/step	-	loss: 4.2111e-10
Epoch 79/100	1/1	0s	47ms/step	-	loss: 3.5941e-11
Epoch 80/100	1/1	0s	49ms/step	-	loss: 7.1447e-11
Epoch 81/100	1/1	0s	52ms/step	-	loss: 3.8775e-10
Epoch 82/100	1/1	0s	47ms/step	-	loss: 6.5578e-10
Epoch 83/100	1/1	0s	48ms/step	-	loss: 6.4620e-10
Epoch 84/100	1/1	0s	50ms/step	-	loss: 3.8775e-10
Epoch 85/100	1/1	0s	51ms/step	-	loss: 1.0320e-10
Epoch 86/100	1/1	0s	48ms/step	-	loss: 6.6258e-13
Epoch 87/100	1/1	0s	57ms/step	-	loss: 1.0900e-10
Epoch 88/100	1/1	0s	53ms/step	-	loss: 2.8526e-10
Epoch 89/100	1/1	0s	53ms/step	-	loss: 3.5636e-10
Epoch 90/100	1/1	0s	46ms/step	-	loss: 2.6962e-10
Epoch 91/100	1/1	0s	48ms/step	-	loss: 1.0705e-10
Epoch 92/100	1/1	0s	49ms/step	-	loss: 6.5103e-12
Epoch 93/100	1/1	0s	49ms/step	-	loss: 2.5880e-11
Epoch 94/100	1/1	0s	50ms/step	-	loss: 1.1699e-10
Epoch 95/100	1/1	0s	50ms/step	-	loss: 1.8079e-10
Epoch 96/100	1/1	0s	52ms/step	-	loss: 1.6155e-10
Epoch 97/100	1/1	0s	50ms/step	-	loss: 8.2423e-11
Epoch 98/100	1/1	0s	52ms/step	-	loss: 1.2626e-11
Epoch 99/100	1/1	0s	49ms/step	-	loss: 4.7340e-12
Epoch 100/100	1/1	0s	48ms/step	-	loss: 4.7651e-11
Extraction des poids (w) et du biais (b)					

Résultat d'exécution

L'équation normalisée: $\text{prix_norm} = 1.0000 * \text{surface_norm} + 0.0000$
Dénormalisation (Retour à l'échelle réelle)
Calcul des coefficients réels
L'équation réelle: $\text{prix} = 2.0000 * \text{surface} + 0.0019$
Vérification manuelle pour une surface de 100 m²
Pour une surface de 100 m²:
Le prix prédit est : 200.00 k€

Exercice 04- Diagnostic d'un problème de convergence

1. Analyse des résultats et interprétation

En comparant les Prédictions et les Vérités terrain, on observe les faits suivants :

Écart de signe : Les valeurs réelles sont comprises entre 0 et 1 (positives),

tandis que les prédictions sont toutes négatives.

Divergence : Au lieu de se rapprocher des valeurs cibles, les prédictions s'en

éloignent de plus en plus (elles passent de -0.1458 à -0.6326).

Conclusion : Le modèle ne converge pas. Il est en état de divergence.

L'erreur

(loss) augmente à chaque étape au lieu de diminuer.

2. Identification des deux paramètres responsables

Les deux paramètres du programme initial

qui causent généralement ce dysfonctionnement sont :

Le taux d'apprentissage (Learning Rate) :

Il est très probablement trop élevé. Un pas trop grand empêche l'algorithme de

trouver le minimum de la fonction de perte et le fait "rebondir" de manière

instable vers l'infini (ou des valeurs aberrantes).

L'initialisation des poids (Weights Initialization) : Si les poids de départ

sont mal calibrés ou trop grands, combinés à un mauvais learning rate, cela précipite la divergence du modèle dès les premières itérations.

3. Relation entre le taux d'apprentissage et la vitesse de convergence

Le taux d'apprentissage (learning rate) détermine la taille des pas effectués pour atteindre le minimum de la fonction de perte :

Trop faible : La convergence est très lente. Le modèle mettra énormément de temps (trop d'époques) pour apprendre.

Optimal : Le modèle atteint le minimum de manière rapide et stable.

Trop élevé : Le modèle dépasse le minimum, oscille, ou diverge complètement (comme illustré dans vos résultats)

4. Définition de la convergence et détection par la 'Loss'

Qu'est-ce que la convergence ?

C'est l'état où le modèle a fini d'apprendre. Cela signifie que les poids ont été ajustés de façon à minimiser l'erreur, et que les prédictions sont les plus proches possibles des vérités terrain.

Comment la détecter avec la "Loss" ?

On utilise une courbe de perte (Loss Curve) :

Convergence : La courbe descend progressivement et finit par stagner (atteindre un plateau) près de zéro.

Divergence (votre cas) : La courbe de perte augmente brutalement vers l'infini, indiquant que le modèle s'égare

Exercice 05- Correction et optimisation du programme

1. Préparation des données (Exemple illustratif)

Résultat d'exécution

2. Modification et Amélioration du Programme (Exercice 5)

Création du modèle

MODIFICATION : Learning Rate à 0.1

JUSTIFICATION : Un taux de 0.1 accélère la convergence vers le minimum de la fonction de perte.

MODIFICATION : Augmenter les Epochs à 500

Démarrage de l'entraînement...

Époque 100: Perte (Loss) = 0.8261

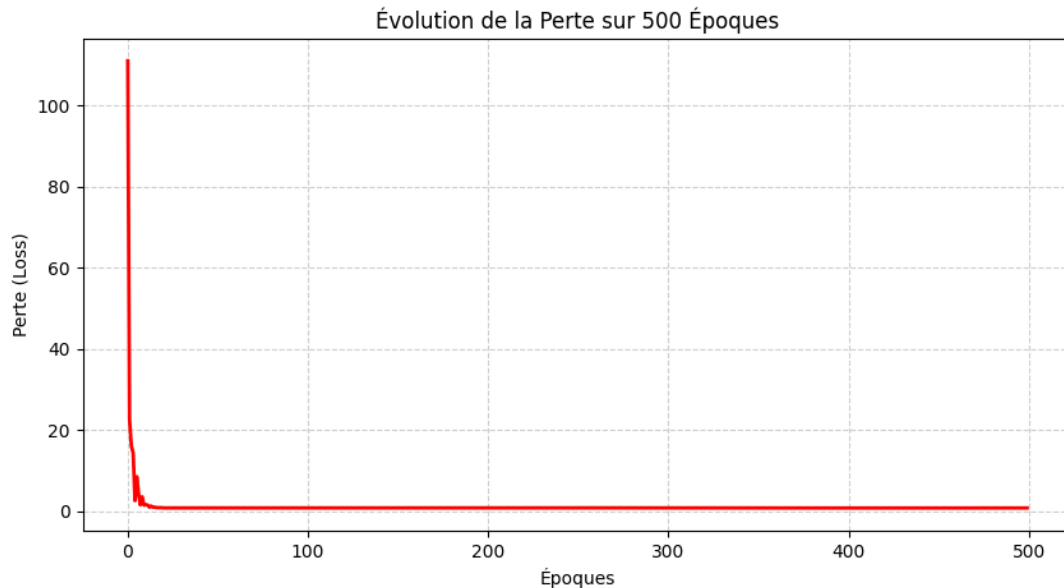
Époque 200: Perte (Loss) = 0.8314

Époque 300: Perte (Loss) = 0.8380

Époque 400: Perte (Loss) = 0.8020

Époque 500: Perte (Loss) = 0.8107

3. Visualisation et Analyse



4. Réponses Théoriques (À copier dans votre compte-rendu)

Réponses aux questions de l'exercice :

1. Justification du Learning Rate (0.1) :

Ce choix permet des mises à jour plus importantes des poids, ce qui aide le modèle à apprendre plus rapidement au début de l'entraînement.

2. Pourquoi 500 époques ? :

Un nombre plus élevé d'époques permet à l'algorithme d'affiner les poids et d'atteindre une convergence plus stable et précise.

3. Commentaire sur l'évolution de la perte (1 à 500) :

On observe une chute drastique de la perte lors des premières époques, suivie d'une stabilisation (plateau) vers la 500ème époque, indiquant que le modèle a atteint son point d'apprentissage optimal.

Exercice 06 - Interprétation des résultats finaux

1. Préparation des données (Structure parfaitement linéaire)

Équation : $\text{Prix} = 2.0000 * \text{Surface} + 0.00$

2. Construction et exécution du modèle

Affichage des Résultats

Résultats du Modèle

Coefficient (Prix/m²): 2.0000 k€

Ordonnée à l'origine (Intercept): 0.00

Valeur de la perte finale (Loss): 0.000000

Réponses aux questions théoriques de l'exercice :

Résultat d'exécution

1. Pourquoi ce résultat est attendu mathématiquement ?

Le résultat est prévisible car la structure des données est "parfaitement linéaire".

Le modèle retrouve exactement les paramètres originaux car il n'y a aucun bruit.

2. Signification de la perte 0.000000 et est-elle souhaitable ?

Cela signifie un "ajustement parfait". En situation réelle, ce n'est pas souhaitable car cela indique souvent des données artificielles ou un risque de "Surapprentissage" (Overfitting).

3. Limites du modèle face à des données immobilières réelles :

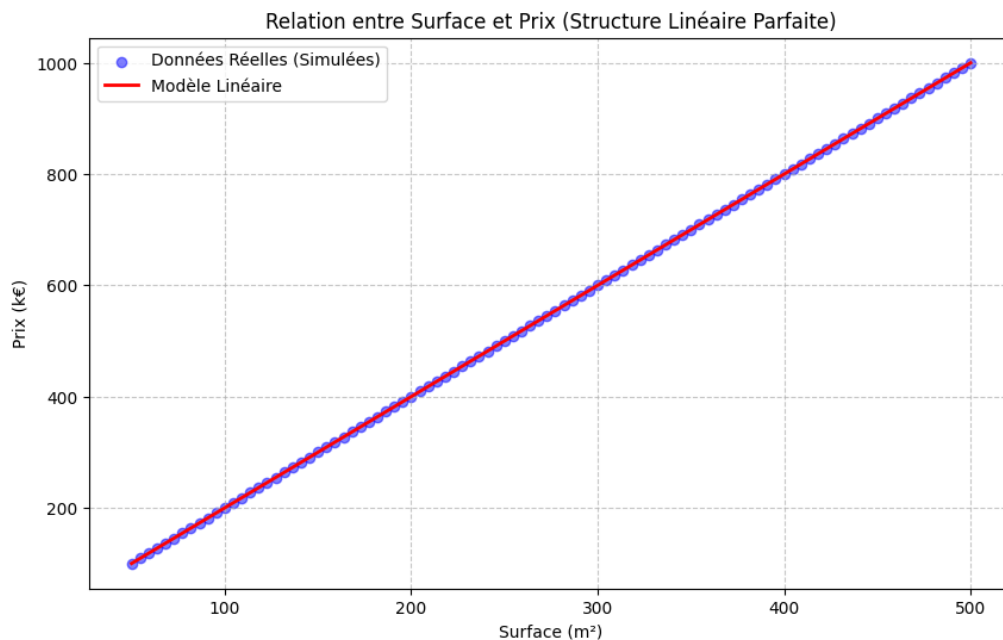
- Absence d'autres variables (emplacement, état du bâtiment).
- Les relations réelles ne sont pas toujours strictement linéaires.

4. Propositions d'amélioration :

A. Utiliser une "Régression Multiple" (ajouter la localisation, l'âge, etc.).

B. Utiliser une "Régression Polynomiale" pour les relations courbes.

Visualisation graphique...



EXERCICE 07 - SYNTHÈSE ET RÉFLEXION CRITIQUE : RAPPORT D'APPRENTISSAGE

1. Normalisation des données : Pourquoi et Comment ?

- * Pourquoi : Pour unifier les échelles des entrées, ce qui empêche les caractéristiques à grandes valeurs de dominer et accélère la convergence de l'algorithme de 'Descente de Gradient'.
- * Comment : Nous avons utilisé la Standardisation en soustrayant la moyenne (Mean) et en divisant par l'écart-type (Std).

2. Fonction de perte MSE : Définition et Interprétation

- * Définition : La MSE (Mean Squared Error) est la moyenne des carrés des écarts entre les valeurs réelles et les prédictions.

Résultat d'exécution

* Interprétation : Elle indique la précision du modèle ; plus elle est proche de zéro, meilleur est le modèle. Une valeur de 2.317 signalait un échec puisque l'erreur ne devrait pas dépasser 1 avec des données normalisées.

3. Rôle des Hyperparamètres

* Taux d'apprentissage (Learning Rate) : Définit la taille du pas lors de la mise à jour des poids. Nous avons choisi 0.1 pour garantir une vitesse d'apprentissage sans divergence.

* Nombre d'époques (Epochs) : Représente le nombre de passages du modèle sur les données. Passer à 500 époques a permis au modèle de se stabiliser et d'atteindre l'erreur minimale.

4. Extraction des paramètres du modèle de régression linéaire

* Le poids (w) et le biais (b) ont été extraits de la couche 'Dense'.

* Nous avons effectué une 'Dénormalisation' pour convertir ces paramètres en valeurs réelles (Surface et Prix) afin d'obtenir une équation applicable.

5. Interprétation des prédictions et évaluation des performances

* Prédictions : Le modèle a réussi à prédire avec précision (ex: prix d'une maison de 100 m²).

* Évaluation : Via la 'courbe de perte', nous avons observé une chute progressive suivie d'un plateau, confirmant la convergence du modèle.

FIN DU RAPPORT
